LB1: Unit-Testing

1 Ausgangslage

Unser Unternehmen wird eine Open-Source-Softwarelösung verwenden, die für unsere geschäftlichen Anforderungen gut geeignet ist. Damit sicher ist, dass alles gut funktioniert, werden zu einzelne Prozeduren Unit-Tests verlangt.

Obwohl die Open-Source-Softwarelösung die notwendigen Grundfunktionen bietet, haben wir spezifische Anforderungen identifiziert, die über die vorhandenen Funktionen hinausgehen. Um diesen Anforderungen gerecht zu werden, beabsichtigen wir, zusätzliche Funktionen mittels Test-Driven-Development selbst zu entwickeln.

2 Aufgabenstellung

2.1 Produkt

Sie schreiben Unit-Test-Code und Programm-Code.
Sie stellen der Lehrperson Ihre Ergebnisse und den funktionierenden Code spätestens am dritten Unterrichtstag vor.

2.2 Sozialform

Die Bearbeitung dieses Auftrags erfolgt in Teams von maximal drei Personen.

2.3 Test-Software

Sie wählen eine Software aus, die Sie für LB1 und LB2 als Basis für Ihre Testumgebung benötigen.

Bedingungen
Die zu testende Software muss folgende Bedingungen erfüllen:
- Die Software besitzt eine implementierte (Geschäfts-)Logik.
- Eine DB-Anbindung ist vorhanden. (Jason und Ähnliches sind nicht erlaubt.)
- Das Framework unterstützt automatisierte Unit-Tests.
- Eine grafische Benutzeroberfläche (UI) ist nicht zwingend nötig.

E-Mail
Wenn Sie Ihre Auswahl getroffen haben, schreiben Sie eine E-Mail an die Lehrperson und CC an Ihren Teampartner mit Betreff "M450 <*Klasse*> Software" und folgenden Inhalt:
- Zusammensetzung des Teams
- Name und Quelle des Programms
- Kurze Beschreibung, was das Programm macht
- Eingesetzte Technologien (Programmiersprache, DB, Entwicklungsumgebung, …)
- Liste der wichtigsten Entitäten (mindestens 3).

3 Die vier Bausteine

Sie erstellen eine Test-Suite, die aus den folgenden **vier Bausteinen** besteht:

3.1 Unit-Test mit Mocking (Abhängigkeiten isolieren)

- **Ziel:** Sie testen eine Code-Unit (Klasse/Methode), die Aufrufe an andere Units oder externe Komponenten (z. B. Datenbank, Repository, externe Services) tätigt.
- **Anforderung:** Alle externen Aufrufe und Abhängigkeiten **müssen gemockt werden**. Die echte Datenbank darf während des Tests nicht kontaktiert werden.
- **Vollständigkeit:** Diese Unit muss **komplett getestet** werden.

3.2 Unit-Test für Logik (Vollständiger Komponenten-Test)

- **Ziel:** Sie testen eine weitere Code-Unit, die eine spezifische, komplexe Logik (z. B. Validierung, Berechnung, Datenverarbeitung) beinhaltet.
- **Anforderung:** Sie testen die Unit auf Herz und Nieren.
- **Vollständigkeit:** Diese Unit muss **komplett getestet** werden.

3.3 Automatisierte Code-Mutation (Beweis der Vollständigkeit)

- **Ziel:** Sie überprüfen die tatsächliche Qualität und Robustheit Ihrer Unit-Tests aus Pflicht 1 und Pflicht 2 mithilfe eines **Mutation-Testing-Tools**.
- **Anforderung:** Führen Sie das Mutation-Testing-Tool über die beiden getesteten Units aus Pflicht 1 und Pflicht 2 aus. Ein sauber generierter Report (z. B. HTML-Datei oder Log-Output) muss vorliegen. Bei der Demonstration erklären Sie der Lehrperson anhand des Reports, welche Mutationen das Tool kreiert hat.
- **Ziel-Kriterium: Es dürfen keine Mutanten überleben!** (Mutation Score = 100% bzw. alle generierten Mutanten wurden von Ihren Tests getötet). Dies dient als eindeutiger Beweis, dass Ihre Unit-Tests aus Pflicht 1 und 2 tatsächlich komplett und lückenlos sind.

3.4 Test-Driven-Development (TDD-Workflow)

- **Ziel:** Sie entwickeln eine neue Funktion nach dem TDD-Prinzip (Red → Green → Refactor), welche einen statistischen Wert aus den Daten der Datenbank berechnet (oder alternativ einen bisher fest programmierten Parameter dynamisch auf Basis von DB-Daten berechnet).
- **Red Phase:** Schreiben Sie zuerst einen Unit-Test für eine noch nicht existierende Berechnungsmethode. Der Test prüft das erwartete Ergebnis der Wertberechnung und schlägt zu Beginn fehl.
- **Green Phase:** Schreiben Sie (oder lassen Sie durch KI generieren) exakt so viel Programmcode, bis der Unit-Test für die Wertberechnung erfolgreich durchläuft.

4 Bewertung

Für die Bewertung zeigen Sie der Lehrperson Ihre Ergebnisse und den funktionierenden Programmcode im
Fachgespräch. Dies können Sie tun, sobald Sie bereit sind, spätestens jedoch am Ende der Lektion in der
dritten Quartalswoche.

Verbindliches Bewertungsraster: Die detaillierte Bewertung erfolgt anhand des separaten Bewertungsrasters M450_LB1_Kriterien.pdf in Google Classroom. Das Raster gliedert sich in folgende
Hauptbereiche:

- Teil 1: Unit-Test mit Mocking (Gewichtung 10)
- Teil 2: Unit-Test für Logik (Gewichtung 10)
- Teil 3: Automatisierte Code-Mutation (Gewichtung 10)
- Teil 4: TDD und Statistische Wertberechnung (Gewichtung 10)
- Automatisierungsgrad der Tests (Gewichtung 5)

Skalierung des **Automatisierungsgrads** (Gewichtung 5):

- 0 Pkt.: Manuell in IDE
- 1 Pkt.: Lokales Skript / Batch
- 2 Pkt.: Zeitgesteuerter Task / Scheduled
- 3 Pkt.: Git-Push / CLI-push Pipeline

Jedes Teilkriterium wird mit 0 bis 3 Punkten bewertet und hat eine festgelegte Gewichtung. Konsultieren Sie für die genauen Teilkriterien und Bewertungsstufen das Dokument **M450_LB1_Kriterien.pdf** in Google Classroom.